PUC Minas



Aula 04 · Front-end

React essencial: componentes, JSX e props

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Escrever **componentes de função** e exportá-los

2

Dominar as regras do **JSX**: `className`, `{}`, fragmentos, tags fechadas

3

Passar dados com **props** e usar `children`

4

Renderizar **listas** com `map` e `key`

5

Aplicar **renderização condicional** e estilos

Antes de começar

- Aula 02: projeto Vite

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Componentes

02 As regras do JSX

03 Props

04 Listas com map e key

05 Renderização condicional

06 Estilos

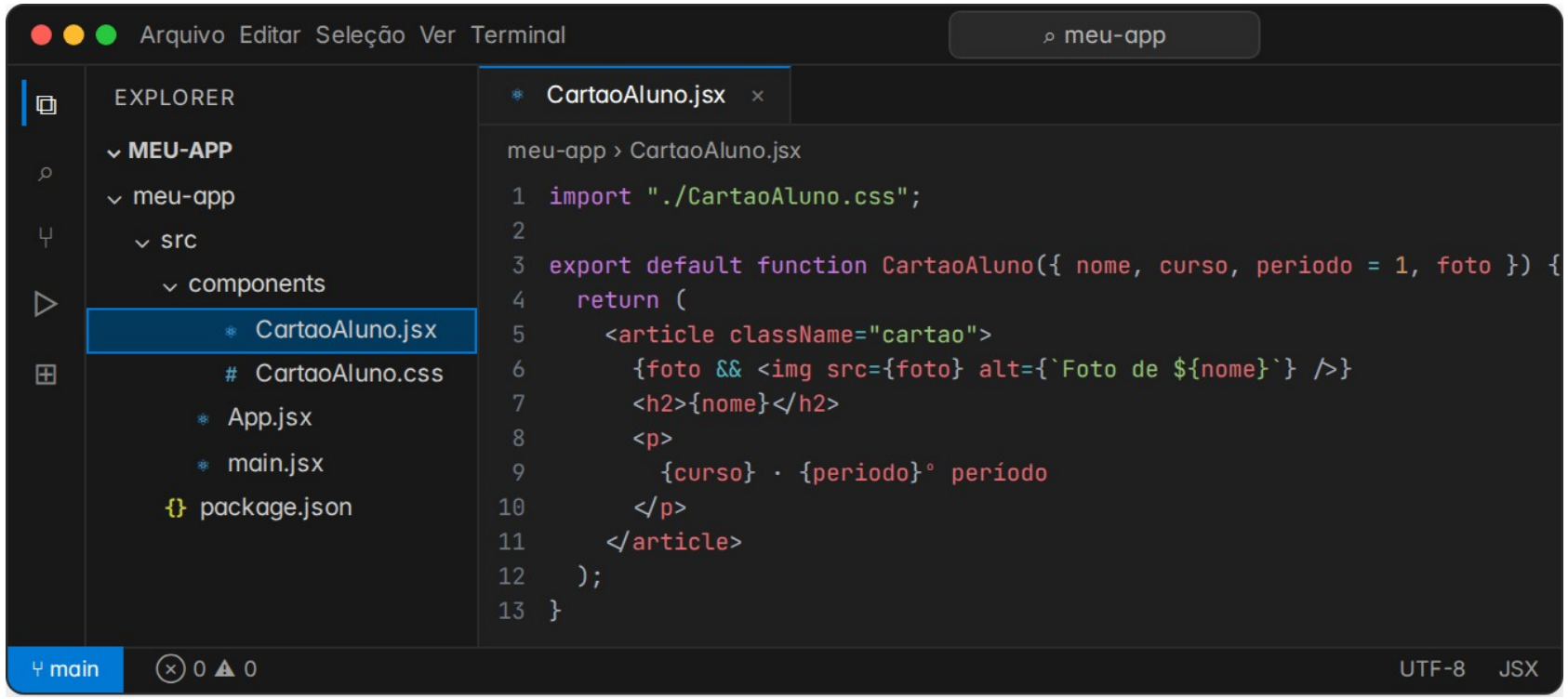
07 Aprofundando: pensando em componentes

01

Componentes

Componentes

Um componente é uma **função JavaScript que devolve JSX**. O nome **começa com letra maiúscula** (é assim que o React diferencia `de uma tag HTML como`). Cada componente fica, por convenção, em seu próprio arquivo.



Componente `CartaoAluno` recebendo dados por *props* (desestruturadas nos parâmetros).

Componentes

src/App.jsx: usando o componente duas vezes

```jsx
import CartaoAluno from "./components/CartaoAluno";

export default function App() {
  return (
    <main>
      <h1>Turma</h1>
      <CartaoAluno nome="Ana Souza" curso="Engenharia de Software" periodo={5} />
      <CartaoAluno nome="Bruno Lima" curso="Ciência da Computação" />
    </main>
  );
}
```

02

As regras do JSX

As regras do JSX

JSX parece HTML, mas é JavaScript. As diferenças que mais causam erro:

HTML	JSX	Por quê
`class="x"`	`className="x"`	`class` é palavra reservada do JS
`for="email"`	`htmlFor="email"`	idem
` `	` `	Toda tag precisa fechar
`onclick="..."`	`onClick={funcao}`	Eventos em *camelCase*, recebem função
`style="color:red"`	`style={{ color: "red" }}`	Objeto JS, propriedades em camelCase
Vários elementos soltos	Um pai ou `<>...</>`	O componente devolve **um** elemento

As regras do JSX

- Dentro de `{ }` vai **qualquer expressão JavaScript**: variáveis, contas, chamadas de função, ternários. Não vale `if` ou `for` (são instruções, não expressões).
- Comentário dentro do JSX: `{/* comentário */}`.

02

As regras do JSX

Aula 04 · Front-end

03

Props

Props

Props (propriedades) são os parâmetros do componente. Fluem **de cima para baixo** (do pai para o filho) e são **somente leitura**: o filho nunca altera uma prop.

Tipos de props e children

```
// strings vão entre aspas; qualquer outro tipo entre chaves
<CartaoAluno nome="Ana" periodo={5} ativo={true} notas={[8, 9.5]} onSelecionar={() => alert("oi")} />

// children: o conteúdo entre as tags de abertura e fechamento
function Painel({ titulo, children }) {
  return (
    <section className="painel">
      <h2>{titulo}</h2>
      {children}
    </section>
  );
}

<Painel titulo="Avisos">
  <p>A prova foi adiada.</p>
</Painel>
```

04

Listas com map e key

Listas com map e key

Renderizando uma lista

```
const alunos = [
  { id: 1, nome: "Ana Souza", curso: "ES" },
  { id: 2, nome: "Bruno Lima", curso: "CC" },
  { id: 3, nome: "Carla Dias", curso: "SI" },
];

export default function ListaAlunos() {
  return (
    <ul>
      {alunos.map((aluno) => (
        <li key={aluno.id}>
          {aluno.nome} ({aluno.curso})
        </li>
      ))}
    </ul>
  );
}
```

Listas com map e key

Atenção

A prop `key` é obrigatória em listas e deve ser **única e estável** (o `id` do banco, o `_id` do MongoDB). Sem ela o console mostra *"Each child in a list should have a unique key prop"*. Evite usar o índice do array (`key={i}`) em listas que mudam de ordem ou têm itens removidos.

05

Renderização condicional

Renderização condicional

Três formas de condicional

```
// ternário: um OU outro
{logado ? <p>Bem-vindo, {nome}</p> : <button>Entrar</button>}

// &&: mostra ou não mostra
{erro && <p className="erro">{erro}</p>}

// retorno antecipado
if (carregando) return <p>Carregando...</p>;
```

Renderização condicional

Erro comum

`{lista.length && }` mostra um **0** na tela quando a lista está vazia, porque `0` é renderizado. Use `{lista.length > 0 && }`.

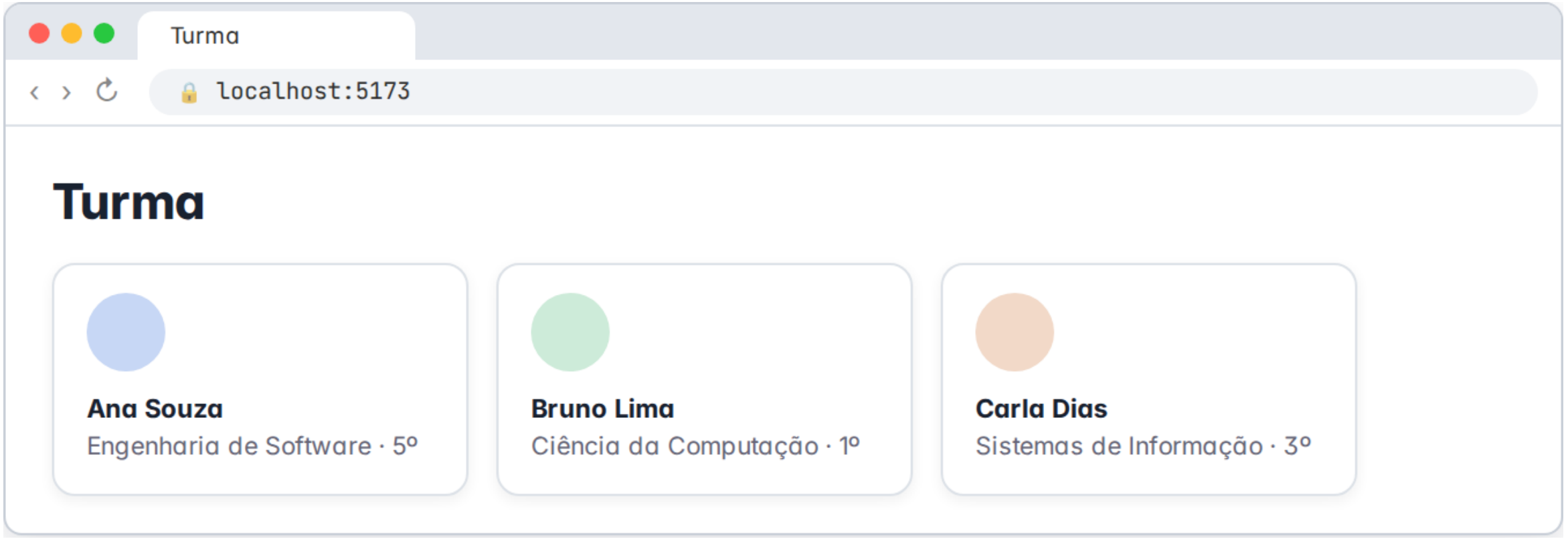
06

Estilos

Estilos

Forma	Exemplo	Quando
CSS por componente	`import "./CartaoAluno.css"`	Padrão dos projetos da disciplina
CSS Modules	`import s from "./Cartao.module.css"` → `className={s.cartao}`	Evita conflito de nomes de classe
Inline	`style={{ marginTop: 8 }}`	Valores dinâmicos pontuais
Bibliotecas	Tailwind, SCSS (projeto `PalpiteDTI`)	Projetos maiores

Estilos



Resultado: o mesmo componente reutilizado três vezes com props diferentes.

07

Aprofundando: pensando em componentes

Aprofundando: pensando em componentes

1. Desenhe a tela (ou pegue o protótipo) e circule cada pedaço que se repete ou tem uma responsabilidade.
2. Dê um nome a cada pedaço: `Cabecalho`, `BarraBusca`, `ListaProdutos`, `CartaoProduto`.
3. Monte a árvore: quem está dentro de quem.
4. Construa primeiro uma versão **estática**, só com props, sem estado.
5. Só depois descubra o que muda (estado) e em qual componente ele deve morar.

Aprofundando: pensando em componentes

Árvore de componentes de uma vitrine

```
App
├── Cabecalho
│   └── Menu
├── BarraBusca
└── ListaProdutos
    └── CartaoProduto  (repetido para cada item)
        └── SeloPromocao  (só quando há desconto)
```

Componente pequeno x componente gigante

Componente gigante

- Um `App.jsx` com 400 linhas
- Difícil de achar onde mexer
- Nada pode ser reaproveitado

vs

Componentes pequenos

- Cada arquivo com até ~100 linhas
- Um nome que explica o que faz
- Reutilizados em várias telas

Props com valores padrão e repasse

Desestruturação, padrão e spread

```
// valor padrão na desestruturação
function Botao({ texto = "Salvar", variante = "primario", ...resto }) {
  return <button className={`btn btn-${variante}`} {...resto}>{texto}</button>;
}

// ...resto repassa onClick, disabled, type etc.
<Botao texto="Excluir" variante="perigo" onClick={excluir} disabled={carregando} />
```

Composição em vez de configuração

Um Card que aceita qualquer conteúdo

```
function Card({ titulo, rodape, children }) {
  return (
    <section className="card">
      <h2>{titulo}</h2>
      <div className="card-corpo">{children}</div>
      {rodape && <footer>{rodape}</footer>}
    </section>
  );
}

<Card titulo="Aluno" rodape={<Botao texto="Editar" />}>
  <p>Ana Souza · Engenharia de Software</p>
</Card>
```

Aprofundando: pensando em componentes

Erro comum de JSX	Correção
``	``
``	``
`onClick={excluir(id)}`	`onClick={() => excluir(id)}`
`style="color: red"`	`style={{ color: "red" }}`
Dois elementos soltos no `return`	Envolver em `<>...`
`{if (x) ...}` dentro do JSX	Ternário `{x ? a : b}` ou `&&`

Convenções dos projetos da disciplina

Uma pasta por componente

`components/PokemonCard/PokemonCard.jsx` + `PokemonCard.css` (projeto `Pokedex`).

Nomes em PascalCase

Arquivo e função com o mesmo nome: `TrainerList.jsx` → `TrainerList`.

export default

Um componente principal por arquivo.

Props descritivas

`onSelecionar`, `emEdicao`, `carregando`: o nome diz o papel.

Aprofundando: pensando em componentes

Na prática

Abra o projeto `Pokedex` e conte os componentes em `src/components`: são mais de 15 peças pequenas (`TypeBadge`, `StatBar`, `SearchBar`…) compostas para formar duas telas. É exatamente a ideia desta aula em um projeto real.

08

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Por que o nome de um componente começa com letra maiúscula?

A Estilo

B Para o React diferenciar de tags HTML

C Exigência do Vite

D Para funcionar no CSS

Resposta

Por que o nome de um componente começa com letra maiúscula?

A Estilo

✓ Para o React diferenciar de tags HTML

C Exigência do Vite

D Para funcionar no CSS

Por quê: seria tratado como tag HTML desconhecida.

Quiz · questão 2 de 5

Pergunta

Em JSX, `class` vira:

A klass

B className

C cssClass

D style

Quiz · questão 2 de 5

Resposta

Em JSX, `class` vira:

A klass

✓ className

C cssClass

D style

Por quê: class é palavra reservada do JavaScript.

Pergunta

Props podem ser alteradas pelo componente filho?

A Sim, sempre

B Não, são somente leitura

C Só strings

D Só com useEffect

Resposta

Props podem ser alteradas pelo componente filho?

A Sim, sempre

✓ Não, são somente leitura

C Só strings

D Só com useEffect

Por quê: Para mudar dados use estado no componente dono.

Pergunta

Qual o problema de `{lista.length && }` com a lista vazia?

A Nenhum

B Mostra um 0 na tela

C Dá erro de compilação

D Some com a página

Resposta

Qual o problema de `{lista.length && }` com a lista vazia?

A Nenhum

✓ Mostra um 0 na tela

C Dá erro de compilação

D Some com a página

Por quê: Use `lista.length > 0 &&`.

Pergunta

O que é obrigatório ao renderizar listas com map?

A useEffect

B Uma prop key única e estável

C Um Fragment

D CSS Modules

Resposta

O que é obrigatório ao renderizar listas com map?

A useEffect

✓ Uma prop key única e estável

C Um Fragment

D CSS Modules

Por quê: Prefira o id do banco ao índice do array.

Fechamento

Resumo da aula

- ✓ Componente = função com nome em **Maiúscula** que devolve JSX; um por arquivo.
- ✓ JSX: `className`, tags sempre fechadas, `{expressão}`, eventos em camelCase, um elemento pai.
- ✓ **Props** passam dados do pai para o filho e são somente leitura; `children` é o conteúdo interno.
- ✓ Listas com `map` e `key` única; condicionais com ternário, `&&` ou `return` antecipado.

Exercícios

1. Crie o componente `Produto` (nome, preço, imagem) e renderize uma vitrine com 6 produtos a partir de um array.

2. Mostre o selo “Promoção” só nos produtos com `desconto > 0`.

3. Crie um componente `Layout` que usa `children` e envolva a vitrine com cabeçalho e rodapé.

4. Encontre e corrija três erros de JSX em um código fornecido pelo professor.

Para ir além

- react.dev/learn/your-first-component
- react.dev/learn/passing-props-to-a-component
- react.dev/learn/rendering-lists

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 04?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web